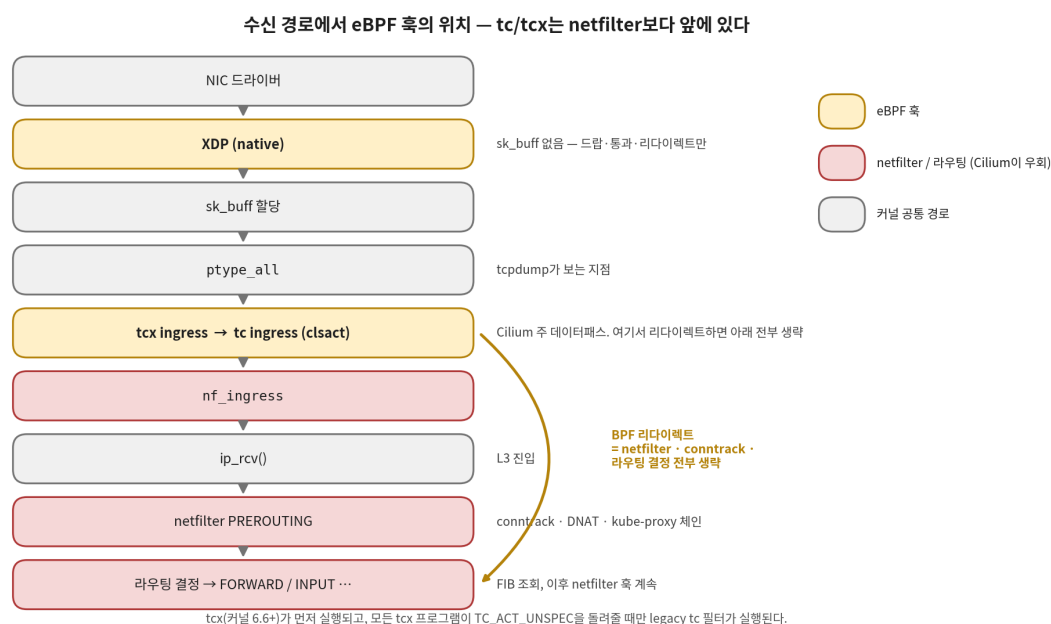


Cilium과 eBPF

- **eBPF** = 커널 안에서 안전하게 실행되는 작은 프로그램. cilium-agent가 커널의 정해진 지점(혹)에 로드
- **BPF 맵** = 그 프로그램이 참조하는 커널 내 테이블. agent가 API 서버를 보고 채움

1. eBPF 혹 지점

1-1. 수신 경로에서의 혹 위치



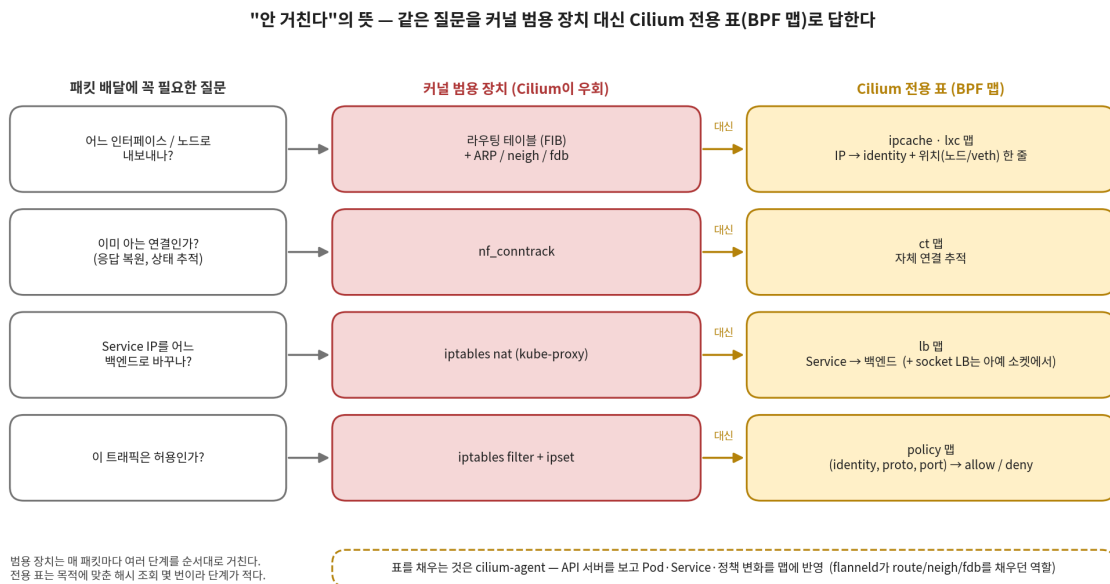
- 커널 수신 경로: XDP → sk_buff 할당 → tcpdump → **tcx / tc ingress** → **ip_rcv()** → **netfilter PREROUTING** → 라우팅 결정
- **tc/tcx 혹은 netfilter보다 앞**. 여기서 리다이렉트하면 conntrack · iptables · 라우팅 결정 전부 생략

혹	커널	sk_buff	Cilium 용도
XDP	4.8+	✗	prefilter(CIDR 차단), NodePort 가속
tc / tcx	tcx 6.6+	✓	주 데이터패스
cgroup/sock	4.17+	✗ (소켓)	socket LB

- tcx: pinned bpf_link 기반. 소유권·순서가 명시적. 커널 미지원 시 legacy tc로 폴백
(`cilium status` → `Attach Mode`)

1-2. 커널 장치 대체

커널 장치 → BPF 맵 대응



- 패킷 배달에 필요한 질문은 그대로. 답하는 장치가 바뀐다

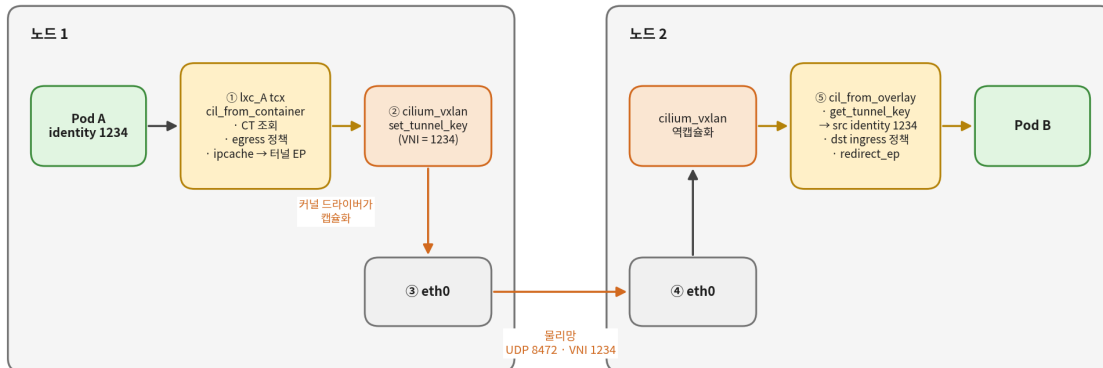
질문	커널 범용 장치	Cilium 전용 표
어디로 내보내나?	라우팅 테이블 + ARP/neigh/fdb	ipcache · lxc 맵 — IP → identity + 위치 한 줄
아는 연결인가?	nf_conntrack	ct 맵
Service → 백엔드?	iptables nat	lb 맵 (socket LB는 소켓에서)
허용인가?	iptables filter + ipset	policy 맵 — (identity, proto, port)

- 표를 채우는 것은 cilium-agent. API 서버를 보고 Pod·Service·정책 변화를 맵에 반영
- 범용 장치는 매 패킷마다 여러 단계를 순서대로 통과. 전용 표는 목적에 맞춘 해시 조회 몇 번 → 단계가 적다

2. 데이터패스

2-1. 터널 모드

터널 모드 — 출발지 Identity가 VXLAN VNI에 실려 간다



Cilium은 VXLAN 헤더를 직접 만들지 않는다 — collect_md 터널 디바이스에 키만 넘기고 커널이 캡슐화한다.

Flannel의 VNI는 논리 네트워크 구분(항상 1). Cilium의 VNI는 보안 Identity. 같은 필드, 완전히 다른 용도.

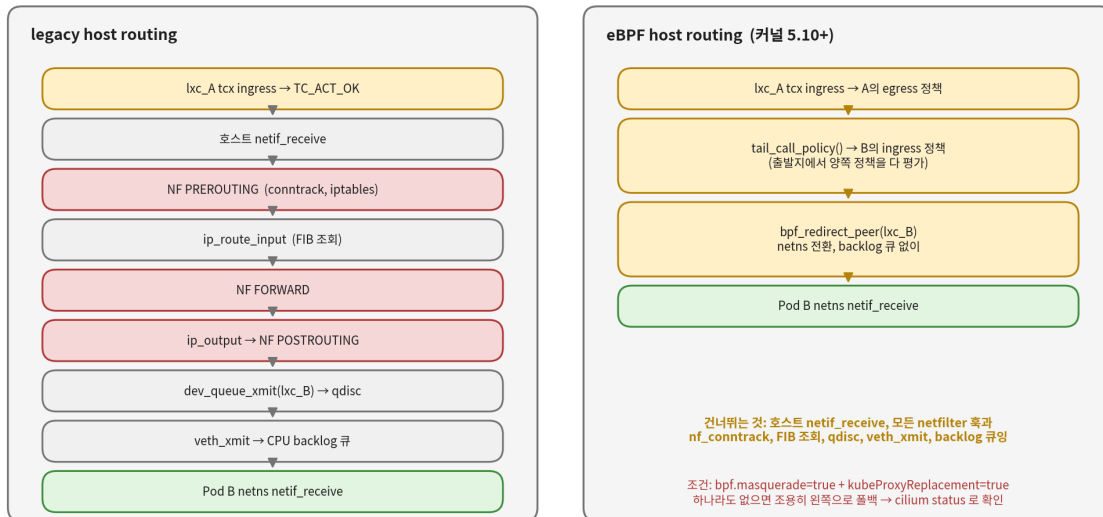
- Pod A → Pod B (VXLAN, UDP 8472)
 - `lxc_A` tcx ingress: CT 조회 → egress 정책 → ipcache 조회(터널 엔드포인트)
 - `bpf_skb_set_tunnel_key()` 로 키 설정 → `cilium_vxlan` 으로 리다이렉트 → 커널 드라이버가 캡슐화
 - 수신 노드 `cil_from_overlay` : `bpf_skb_get_tunnel_key()` 로 src identity 복원 → ingress 정책 → Pod B
- Cilium은 VXLAN 헤더를 직접 만들지 않음. collect_md 터널 디바이스에 키만 전달

```
key.tunnel_id = get_tunnel_id(seclabel); /* identity → VNI */
```

- Flannel VNI = 논리 네트워크 구분(항상 1). Cilium VNI = **출발지 보안 Identity**

2-2. eBPF host routing

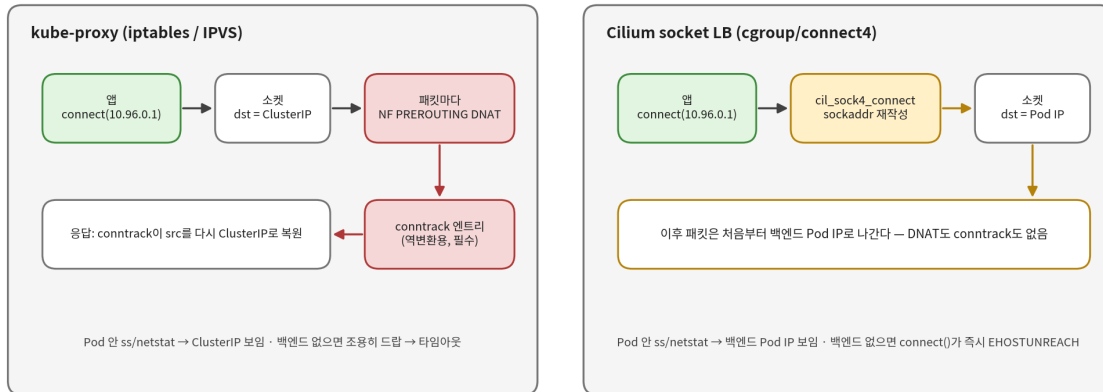
같은 노드 Pod A → Pod B — legacy 경로 vs eBPF host routing



- 헬퍼 두 개, 커널 5.10+
 - `bpf_redirect_peer()` — veth peer netns로 ingress → ingress 직행. CPU backlog 큐 없이
 - `bpf_redirect_neigh()` — FIB 조회 + L2 주소 채워 NIC로 직행
- 건너뛰는 것: 호스트 `netif_receive`, 모든 `netfilter` 혹은 `nf_conntrack`, FIB 조회, `qdisc`, `veth_xmit`, backlog 큐잉
- 정책 평가 위치 변화: 목적지 netns로 직행하므로 **출발지에서 egress + ingress 정책 모두 평가**
- 활성화 조건: `bpf.masquerade=true` and `kubeProxyReplacement=true`. 하나라도 없으면 조용히 legacy로 폴백
 - 확인: `cilium status` → Routing: Host: BPF / Legacy

3. kube-proxy vs socket LB

Service 접속 — kube-proxy는 패킷마다, socket LB는 connect() 한 번



NodePort 등 NIC로 들어오는 외부 트래픽은 소켓이 없으므로 tc LB(패킷 단위 DNAT)로 처리한다.

- kube-proxy: 패킷마다 DNAT. socket LB: `connect()` 시점에 소켓의 목적지 주소 재작성

```
__section("cgroup/connect4")
int cil_sock4_connect(struct bpf_sock_addr *ctx) { ... }
ctx->user_ip4 = backend->address; /* sockaddr 자체를 고친다 */
```

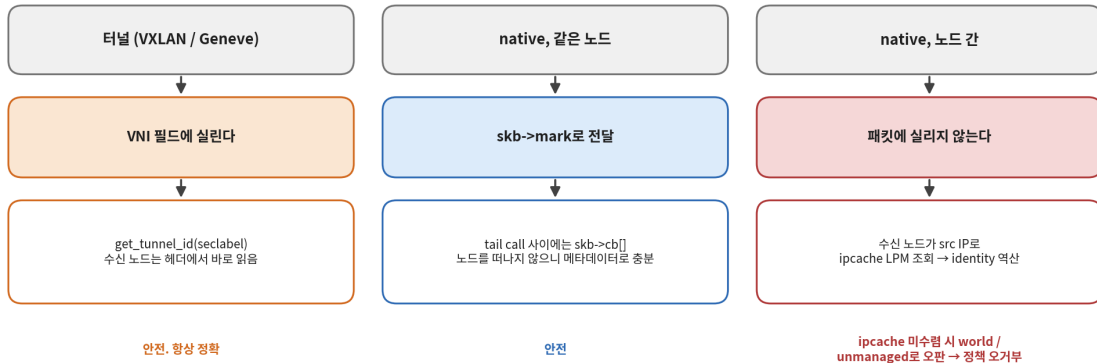
	kube-proxy	socket LB
변환 시점	패킷마다	<code>connect()</code> 1회
conntrack	역변환용 필수	불필요
Pod 안에서 본 peer	ClusterIP	백엔드 Pod IP
백엔드 없을 때	조용히 드랍 → 타임아웃	<code>connect()</code> 즉시 <code>EHOSTUNREACH</code>

- NodePort 등 NIC로 들어오는 외부 트래픽은 소켓이 없으므로 **tc LB**(패킷 단위 DNAT + 자체 CT 맵)
- DSR 제약: **DSR + VXLAN 불가**. 터널 모드에서 DSR은 Geneve만
- `kubeProxyReplacement` 값: 1.15 이하 `strict / probe`, 1.16+ `true / false`. 옛 값은 거부됨

4. Identity와 정책

4-1. Identity 전달

출발지 Identity는 어떻게 목적지까지 가는가 — 모드마다 다르다



운영 환경은 세 번째. ClusterMesh나 노드 추가 직후 Hubble에 world 로 보이며 거부되면 ipcache 수렴을 의심한다.

전송	운반 수단
터널 (VXLAN / Geneve)	VNI 필드
native, 같은 노드	<code>skb->mark</code>
native, 노드 간	패킷에 실리지 않음 → 수신 노드가 src IP로 ipcache 조회해 역산

- 함정: native routing에서 ipcache 미수렴 시 Hubble에 `world` (2) / `unmanaged` (3)로 표시, 정책 오거부. ClusterMesh·노드 추가 직후 흔함

4-2. 정책 맵

- 엔드포인트마다 `cilium_policy_v3_<ep>`. **LPM_TRIE**, 기본 **16384** 엔트리

```

type PolicyKey struct {
    Prefixlen      uint32    // LPM - 프로토콜 + 포트 범위를
    프리픽스로
    Identity       uint32    // ← IP가 아니라 identity
    TrafficDirection uint8
    Nexthdr       uint8
    DestPortNetwork uint16
}
  
```

- 값: `ProxyPortNetwork` ($\neq 0$ → Envoy/DNS 프록시), `Flags` bit0 = Deny
- 오버플로: 맵이 가득 차면 기본은 **새 엔트리 조용히 설치 실패(fail-open)**. `-enable-endpoint-lockdown-on-policy-overflow`로 fail-closed. 기본 off

4-3. L7 정책

- 기본: `mark` 설정 후 **iptables TPROXY** 룰에 의존. `bpf.tproxy=true` 면 `bpf_sk_assign()` 으로 iptables 없이 Envoy 부착 (netkit 비호환)
- **L7 가시성 = L7 정책**. `proxy-visibility` 어노테이션 제거됨. `rules: http: [{}]` 규칙이 있어야 관측되고, 그 규칙은 동시에 집행

5. 폴백 동작

기능	조건 미충족 시	<code>cilium status</code> 확인 행
eBPF host routing	<code>bpf.masquerade</code> 또는 KPR 꺼짐 → legacy	<code>Routing: Host:</code>
tcx	커널 < 6.6 → legacy tc	<code>Attach Mode:</code>
socket LB	cgroup v2 없음 → 미동작	<code>Socket LB Coverage:</code>

- 기대보다 낮은 성능의 원인 대부분. 설치 후 반드시 확인
- tcpdump: 들어오는 패킷은 잡힘(`ptype_all` 이 tc보다 앞). **BPF 리다이렉트 이후 경로는 안 잡힘**

6. 요약

1. tc/tcx 혹은 **netfilter보다 앞**. 리다이렉트하면 conntrack · iptables · 라우팅 생략
2. host routing: 9단계 → 4단계. 단 **masquerade + KPR 둘 다 필요**, 아니면 조용히 폴백
3. socket LB: `connect()` 시점 변환. conntrack 불필요, 실패는 `EHOSTUNREACH`
4. Identity 전달: 터널은 **VNI**, native 노드 간은 **ipcache 역산** → 수렴 문제 = 정책 오류
5. 정책 맵 오버플로 기본 **fail-open**. L7 관측은 정책 없이 불가